Experiment: Student Management System Using Structures and Function Categories

Aim: To write and implement a **C program using structures, arrays of structures, structure pointers, and different categories of functions based on arguments and return values**.

Problem Statement: Write a C program to implement a **Student Management System** for 3 students using the following structure:

```
struct Student
{
   int roll;
   char name[30];
   float marks1;
   float marks2;
   float marks3;
   float average;
};
```

The program must demonstrate:

1. Function with **no arguments and no return value**.
2. Function with **no arguments and a return value**.
3. Function with **arguments and no return value**.
4. Function with **arguments and a return value**.
5. Passing an **array of structures** to a function.
6. Passing a **structure using a pointer**.
7. Returning a **pointer to a structure**.
8. Accessing structure members using both . and -> operators.

Structure Attributes

Attribute	Data Type	Description
roll	int	Student roll number
name	char[30]	Student name
marks1	float	Marks in Subject 1
marks2	float	Marks in Subject 2
marks3	float	Marks in Subject 3
average	float	Average of three subjects

Function Requirements

Function	Category / Concept
displayTitle()	No argument, no return
inputStudent()	No argument, returns structure
calculateAverage(s)	Argument, returns value
displayStudent(s)	Argument, no return
displayAll(s, n)	Array of structures
updateMarks(&s[0])	Structure pointer
getStudent(s)	Returns structure pointer

Algorithm

1. Start the program.
2. Define the Student structure with roll number, name, three marks, and average.
3. Define displayTitle() to display the program title.
4. Define inputStudent() to read student details and return a structure.
5. Define calculateAverage() to calculate and return the average marks.
6. Define displayStudent() to display one student's details.
7. Define displayAll() to accept an array of structures and display all students.
8. Define updateMarks() to accept a structure pointer and add 5 bonus marks to Subject 1.
9. Recalculate the student's average after updating the marks.
10. Define getStudent() to return the address of the first structure in the array.
11. In main(), create an array of 3 Student structures.
12. Read the details of all three students.
13. Calculate the average for each student.
14. Display all students before modification.
15. Pass the address of the first student to updateMarks().
16. Display all students after modification.
17. Call getStudent() and store the returned address in a structure pointer.
18. Display the returned student's details using the -> operator.
19. Pass another complete structure to displayStudent().
20. Stop.

Output:

```
========================================
     STUDENT MANAGEMENT SYSTEM
========================================
Enter details of 3 students:
--- Student 1 ---
Enter Roll Number : 101
Enter Name        : Rahul
Enter Marks 1     : 80
Enter Marks 2     : 85
Enter Marks 3     : 90

--- Student 2 ---
Enter Roll Number : 102
Enter Name        : Anu
Enter Marks 1     : 90
Enter Marks 2     : 92
Enter Marks 3     : 88

--- Student 3 ---
Enter Roll Number : 103
Enter Name        : Ravi
Enter Marks 1     : 75
Enter Marks 2     : 80
Enter Marks 3     : 85


BEFORE UPDATING MARKS
============================================================
Roll    Name          M1      M2      M3      Average
============================================================
101     Rahul         80.00   85.00   90.00   85.00
```

102	Anu	90.00	92.00	88.00	90.00
103	Ravi	75.00	80.00	85.00	80.00

Adding 5 bonus marks to Student 1...

AFTER UPDATING MARKS

Roll	Name	M1	M2	M3	Average
101	Rahul	85.00	85.00	90.00	86.67
102	Anu	90.00	92.00	88.00	90.00
103	Ravi	75.00	80.00	85.00	80.00

STUDENT RETURNED USING POINTER
Roll Number : 101
Name : Rahul
Marks 1 : 85.00
Average : 86.67

PASSING COMPLETE STRUCTURE
Roll Number : 102
Name : Anu
Marks 1 : 90.00
Marks 2 : 92.00
Marks 3 : 88.00
Average : 90.00

Experiment: Interpolation Search Using Different Categories of Functions

Develop a C program to implement Interpolation Search on a sorted array using user-defined functions. The program must demonstrate all four categories of functions:

1. **No arguments and no return value** – display the menu.
2. **Arguments and no return value** – display the array.
3. **No arguments and return value** – read and return the search key.
4. **Arguments and return value** – perform Interpolation Search and return the position of the searched element.

The program should display whether the search element is found and its position.

Interpolation Search Formula

The probable position is calculated as:

Where:

- low – lower index
- high – higher index
- key – element to be searched
- A[low] – value at the lower index
- A[high] – value at the higher index
- pos – estimated position

Example:

Given the sorted array:

10 20 30 40 50 60 70 80 90

0 1 2 3 4 5 6 7 8

Search key = 70

Therefore:

A[6] = 70

Example Output

```
Enter number of elements: 9

Enter elements in sorted order:
10 20 30 40 50 60 70 80 90

===== INTERPOLATION SEARCH =====
1. Display Array
2. Search Element
3. Exit

Enter choice: 1

Array: 10 20 30 40 50 60 70 80 90

===== INTERPOLATION SEARCH =====
1. Display Array
2. Search Element
3. Exit

Enter choice: 2
Enter element to search: 70
Element found at index 6
```

```
===== INTERPOLATION SEARCH =====
1. Display Array
2. Search Element
3. Exit

Enter choice: 2

Enter element to search: 75

Element not found

Enter choice: 3

Program terminated.
```